

Identificação e Correção de Gargalos de Level Design via Aprendizado por Reforço Profundo

Robert Ronald Calla Maguiña
Departamento de Informática, PUC-Rio
Rio de Janeiro, RJ, Brasil

Abstract

Este trabalho relata o andamento do desenvolvimento de um sistema de Inteligência Artificial voltado à identificação e correção automática de gargalos de *level design* em fases 2D baseadas em física. O ambiente de desenvolvimento é estruturado na engine Godot 4.7, com foco no processamento de mecânicas multiplayer. Diferentemente de abordagens tradicionais de navegação (*pathfinding*), o objetivo é treinar um agente capaz de analisar a topologia do mapa, identificar zonas de estrangulamento que prejudicam a fluidez do jogo e aplicar técnicas de geração de conteúdo para sugerir adaptações no cenário. Propomos uma arquitetura que combina a extração estruturada do ambiente em uma matriz 2D, o processamento espacial por meio de Redes Neurais Convolucionais (CNN) e um agente de Aprendizado por Reforço Profundo (DRL) inspirado em conceitos de *Curiosity-Driven Exploration* e de *Procedural Content Generation via Reinforcement Learning* (PCGRL). Formalizamos o problema como um Processo de Decisão de Markov, detalhamos as decisões de projeto para cada componente do pipeline (representação de estado, extrator convolucional, política e função de recompensa híbrida intrínseca/extrínseca), descrevemos o estado atual da implementação e propomos uma metodologia de avaliação – com métricas, baselines e protocolo experimental – para os próximos ciclos do projeto. Discutimos ainda as limitações e os riscos de *reward hacking* inerentes à abordagem, bem como um roteiro de trabalhos futuros.

Keywords

Level Design, Aprendizado por Reforço Profundo, Geração Procedural de Conteúdo, Curiosity-Driven Exploration, Godot Engine, Playtesting Automatizado, Redes Neurais Convolucionais

1 Introdução

O *level design* é um dos pilares da experiência de jogo: a forma como corredores, salas, obstáculos e rotas alternativas são dispostos determina diretamente o ritmo, a dificuldade percebida e a fluidez com que o jogador avança pela fase. Contudo, identificar manualmente pontos de fricção que prejudicam essa fluidez – os chamados *gargalos* (*bottlenecks*) – é um processo custoso, subjetivo e dependente de extensas sessões de *playtesting* humano. Em produções com ciclos de iteração curtos, ou em fases geradas proceduralmente, esse custo se torna ainda mais crítico, pois cada nova versão do nível exigiria uma nova rodada de testes manuais.

Este trabalho relata o progresso da pesquisa e desenvolvimento de uma aplicação de Inteligência Artificial para identificação e correção desses gargalos. O ambiente de desenvolvimento está sendo estruturado na engine Godot 4.7 [2], com foco no processamento de fases 2D baseadas em física e mecânicas multiplayer, um domínio no qual gargalos de navegação afetam não apenas um jogador isolado, mas a dinâmica de disputa por espaço entre múltiplos agentes.

1.1 Motivação

A motivação central do projeto é que ferramentas automáticas de *playtesting* baseadas em IA podem reduzir o custo de iteração do processo de *level design*, permitindo que designers recebam retroalimentação quantitativa sobre a qualidade estrutural de um nível antes mesmo de expô-lo a testadores humanos. Diferente de heurísticas estáticas de análise de grafos (por exemplo, medidas de centralidade sobre o grafo de navegabilidade), um agente treinado por reforço pode capturar efeitos dinâmicos do *gameplay* – como congestionamento causado por múltiplos jogadores disputando o mesmo corredor – que não são triviais de expressar como regra fixa.

1.2 Definição do Problema

O objetivo do projeto é ir além da simples navegação (*pathfinding*). A meta é treinar um agente de aprendizado de máquina capaz de:

- (1) analisar a topologia do mapa a partir de uma representação estruturada e independente de renderização gráfica;
- (2) identificar zonas de estrangulamento que prejudicam a fluidez do jogo, isto é, regiões do mapa em que o fluxo de travessia entre pontos de interesse é anormalmente baixo; e
- (3) aplicar técnicas de geração de conteúdo para sugerir adaptações no cenário que aliviem esses gargalos, fechando o ciclo entre diagnóstico e correção.

Em outras palavras, o agente não deve apenas percorrer o nível, mas aprender a apontar – e eventualmente corrigir – falhas estruturais de *design*.

1.3 Contribuições

As contribuições deste relatório de andamento são:

- uma arquitetura de representação de ambiente independente de renderização gráfica, baseada em matrizes 2D extraídas diretamente do *TileMap* e do *NavigationServer2D* do Godot;

- uma formalização do problema como um Processo de Decisão de Markov (MDP), com espaço de estados, espaço de ações e uma proposta de função de recompensa híbrida (intrínseca + extrínseca);
- uma proposta de pipeline *CNN + DRL* para transformar essa representação em decisões de ação sobre o nível, incluindo a escolha justificada do algoritmo de otimização de política e do módulo de curiosidade;
- uma metodologia de avaliação planejada, com métricas, baselines e protocolo experimental, ainda a ser executada nos próximos ciclos do projeto; e
- uma discussão inicial sobre limitações e riscos metodológicos, notadamente o risco de *reward hacking*.

1.4 Organização do Documento

O restante deste artigo está organizado da seguinte forma. A Seção 2 discute trabalhos relacionados em geração procedural de conteúdo via aprendizado por reforço, *playtesting* automatizado e exploração guiada por curiosidade. A Seção 3 formaliza o problema como um MDP. A Seção 4 detalha a arquitetura do sistema proposto. A Seção 5 descreve o estado atual da implementação. A Seção 6 propõe a metodologia de avaliação. A Seção 7 discute limitações e riscos. A Seção 8 discute considerações éticas e de uso responsável. A Seção 9 apresenta o roteiro de trabalhos futuros, e a Seção 10 conclui o artigo.

2 Trabalhos Relacionados

O uso de aprendizado por reforço para geração e avaliação de conteúdo de jogos vem se consolidando como uma alternativa aos métodos tradicionais de geração procedural baseados em regras ou busca (*search-based PCG*) [8]. Esta seção revisa três frentes que sustentam a arquitetura proposta: geração procedural via aprendizado por reforço, *playtesting* automatizado com agentes humanoides, e exploração guiada por curiosidade.

2.1 Geração Procedural de Conteúdo via Aprendizado por Reforço

Khalifa et al. [3] propõem o PCGRL (*Procedural Content Generation via Reinforcement Learning*), no qual a geração de níveis é formulada como um problema sequencial de decisão: um agente edita o mapa tile a tile e recebe recompensas conforme métricas de qualidade do nível resultante (por exemplo, conectividade, número de inimigos alcançáveis ou tamanho do caminho mais curto). Diferentemente de modelos generativos treinados por aprendizado supervisionado sobre corpora de níveis existentes, o PCGRL não depende de grandes conjuntos de dados anotados, e sim de uma função de recompensa bem definida sobre o espaço de níveis – uma característica central que buscamos reaproveitar neste projeto. Essa formulação inspira diretamente a representação de estado e o espaço de ação discutidos na Seção 4 deste trabalho.

2.2 Playtesting Automatizado com Agentes Humanoides

Na frente de *playtesting* automatizado, De Woillemont et al. [1] utilizam aprendizado por reforço para gerar agentes com estilos de jogo semelhantes aos humanos, permitindo testar um jogo sob perfis de comportamento variados sem depender de testadores humanos para cada iteração. Essa linha de trabalho parte da premissa de que um agente puramente ótimo (por exemplo, um agente que sempre encontra o caminho mais curto) não é representativo do comportamento real de jogadores, que exploram, hesitam e cometem erros. Sestini et al. [7] avançam essa linha propondo trajetórias proximais condicionadas por curiosidade (*curiosity-conditioned proximal trajectories*) para validação automatizada de *gameplay*, combinando exploração guiada por curiosidade com métodos de otimização de política por região de confiança para detectar estados de jogo raros ou problemáticos (por exemplo, situações de *softlock* ou exploração de *bugs*).

2.3 Exploração Guiada por Curiosidade

O componente de exploração guiada por curiosidade utilizado neste projeto segue a formulação do *Intrinsic Curiosity Module* (ICM) proposta por Pathak et al. [5], na qual um modelo preditivo aprende a antecipar a representação latente do próximo estado a partir do estado e da ação atuais; o erro dessa predição é utilizado como recompensa intrínseca, incentivando o agente a visitar regiões do ambiente cuja dinâmica ainda não foi bem modelada. Esse mecanismo é particularmente adequado ao nosso cenário, pois gargalos de *level design* tendem a corresponder a regiões pouco visitadas por políticas ingênuas de navegação, exatamente as regiões que um sinal de curiosidade tende a priorizar.

2.4 Posicionamento deste Trabalho

A Tabela 1 resume o posicionamento do presente trabalho em relação aos três trabalhos discutidos. O projeto se posiciona na interseção dessas linhas: assim como o PCGRL, utiliza uma representação matricial do nível como estado de um processo de decisão de Markov; e, assim como os trabalhos de *playtesting* automatizado, incorpora exploração orientada por curiosidade – não para gerar estilos de jogador, mas para que o próprio agente descubra e sinalize regiões do mapa com baixo fluxo de travessia, tratando a detecção de gargalos como o objetivo central da recompensa, e não apenas como um subproduto da exploração.

3 Formalização do Problema

3.1 Gargalos de Level Design

Definimos informalmente um gargalo como uma região do nível cujo fluxo de travessia – o número de vezes que agentes passam por aquela região durante um episódio de *gameplay*, normalizado pelo tempo total do episódio – é significativamente menor do que o esperado para uma região com conectividade equivalente no grafo de navegação. Essa

Table 1: Posicionamento em relação aos trabalhos relacionados

Trabalho	Foco principal
PCGRL [3]	Geração de níveis tile a tile via RL, recompensa sobre métricas globais de qualidade do nível.
De Woillemont et al. [1]	Geração de agentes de <i>playtesting</i> com estilos de jogo humanóides.
Sestini et al. [7]	Validação automatizada de <i>gameplay</i> via exploração condicionada por curiosidade.
Este trabalho	Diagnóstico e sinalização de gargalos de <i>level design</i> como objetivo central da recompensa, combinando representação matricial (PCGRL) com curiosidade (ICM) para priorizar regiões de baixo fluxo.

definição captura tanto gargalos “estruturais” (um corredor estreito que fisicamente limita a passagem simultânea de múltiplos jogadores) quanto gargalos “comportamentais” (uma bifurcação que jogadores tendem a evitar por parecer, subjetivamente, menos vantajosa).

3.2 Processo de Decisão de Markov

Modelamos o problema como um Processo de Decisão de Markov (MDP) $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, no qual:

- $\mathcal{S}$ é o espaço de estados, definido pela matriz 2D $G \in \{0, 1\}^{H \times W}$ que representa o nível (Seção 3.3), possivelmente aumentada por canais adicionais que codificam a posição do agente e de pontos de interesse (entrada, saída, itens);
- $\mathcal{A}$ é o espaço de ações, detalhado na Seção 3.4;
- $P(s_{t+1} | s_t, a_t)$ é a dinâmica de transição, determinada pelo motor de física e navegação do Godot;
- $R(s_t, a_t, s_{t+1})$ é a função de recompensa, detalhada na Seção 3.5; e
- $\gamma \in [0, 1]$ é o fator de desconto.

O objetivo do treinamento é encontrar uma política $\pi_\theta(a_t | s_t)$, parametrizada por uma rede neural com parâmetros θ , que maximize o retorno esperado

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t, s_{t+1}) \right]. \quad (1)$$

3.3 Espaço de Estados

O estado observado pelo agente em cada instante t é a matriz G_t descrita na Seção 4.2, processada por um extrator convolucional $\phi(\cdot)$ (Seção 4.3) que produz um vetor de características latente $z_t = \phi(G_t) \in \mathbb{R}^d$. Esse vetor latente é a entrada tanto da rede de política quanto do módulo de curiosidade.

3.4 Espaço de Ações

Propomos um espaço de ações híbrido, refletindo a natureza em duas fases do problema (navegação e, posteriormente, edição de conteúdo):

- **Ações de navegação:** um vetor de deslocamento discreto $a^{\text{nav}} \in \{\text{norte, sul, leste, oeste, parado}\}$, compatível com a granularidade da matriz de *tiles*;
- **Ações de sinalização/edição** (fase posterior do projeto): $a^{\text{edit}} \in \{\text{nenhuma, marcar gargalo, remover bloqueio, adicionar}\}$, aplicadas à célula da matriz correspondente à posição atual do agente.

Na fase atual do projeto, apenas a^{nav} está em uso; o espaço de edição será incorporado progressivamente, conforme discutido na Seção 9.

3.5 Função de Recompensa

Propomos uma função de recompensa híbrida, combinando um termo extrínseco r_t^{ext} , responsável por penalizar gargalos, e um termo intrínseco r_t^{int} , responsável por guiar a exploração:

$$R(s_t, a_t, s_{t+1}) = r_t^{\text{ext}} + \beta r_t^{\text{int}}, \quad (2)$$

em que β é um coeficiente de balanceamento entre os dois termos.

O termo extrínseco penaliza células de baixo fluxo de travessia. Seja $\tau(v)$ o fluxo acumulado de travessia da célula v ao longo de uma janela de N episódios, e θ_τ um limiar abaixo do qual uma célula é considerada um gargalo. Definimos

$$r_t^{\text{ext}} = - \sum_{v \in \mathcal{V}(s_t)} w(v) \cdot \mathbb{K}[\tau(v) < \theta_\tau], \quad (3)$$

em que $\mathcal{V}(s_t)$ é a vizinhança imediata da posição do agente em s_t e $w(v)$ é um peso proporcional à centralidade da célula v no grafo de navegação – de modo que gargalos em rotas centrais sejam penalizados mais fortemente do que gargalos em regiões marginais do mapa.

O termo intrínseco segue a formulação do ICM [5]: um modelo preditivo f_ψ estima a representação latente do próximo estado, $\hat{z}_{t+1} = f_\psi(z_t, a_t)$, e o erro de predição é utilizado como recompensa,

$$r_t^{\text{int}} = \frac{\eta}{2} \|\hat{z}_{t+1} - z_{t+1}\|_2^2, \quad (4)$$

em que η é uma constante de escala. A intuição é que regiões do nível cuja dinâmica é mal modelada pelo agente – tipicamente regiões pouco visitadas, como gargalos – geram maior erro de predição e, portanto, maior recompensa intrínseca, complementando o sinal extrínseco da Equação (3) durante as fases iniciais do treinamento, em que as estimativas de $\tau(v)$ ainda são ruidosas.

3.6 Exemplo Ilustrativo

Para tornar concreta a formalização anterior, considere o corredor hipotético representado na Figura 1: uma matriz 5×3 na qual a célula central da coluna do meio é o único ponto de passagem entre a metade esquerda e a metade direita do nível.

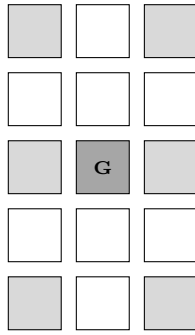


Figure 1: Exemplo ilustrativo de gargalo estrutural: células cinza-claras são paredes, células brancas são caminho livre, e a célula cinza-escura marcada com “G” é o único ponto de passagem entre as duas metades do nível.

Nesse exemplo, um agente de navegação ingênuo (por exemplo, treinado apenas com recompensa esparsa por atingir a saída) tende a atravessar a célula **G** com a mesma frequência que qualquer outra célula do caminho, uma vez que ela é, por definição, o único caminho possível – não há, portanto, sinal de que essa célula seja estruturalmente diferente das demais apenas observando a frequência de visitação. É por isso que a Equação (3) pondera o fluxo $\tau(v)$ pela centralidade $w(v)$ da célula no grafo de navegação: a célula **G** possui centralidade de intermediação (*betweenness centrality*) muito mais alta do que as demais células do corredor, pois todo caminho entre as duas metades do nível necessariamente passa por ela. Um valor de $\tau(v)$ comparável ao das demais células livres, combinado com um peso $w(v)$ elevado, é o que sinaliza **G** como um gargalo estrutural relevante, e não apenas mais uma célula de passagem. Em corredores com múltiplos caminhos alternativos, por outro lado, a centralidade de cada célula individual tende a ser menor, e um gargalo só emerge quando o fluxo observado $\tau(v)$ cai significativamente abaixo do esperado para células de centralidade comparável – é essa combinação entre topologia estática ($w(v)$) e comportamento dinâmico observado ($\tau(v)$) que buscamos capturar com a função de recompensa proposta, e que uma heurística puramente estrutural (que considerasse apenas $w(v)$) não conseguiria distinguir sozinha.

4 Arquitetura do Sistema

A Figura 2 ilustra o pipeline proposto, do ambiente Godot até a política de ação, passando pelos três componentes descritos a seguir.

4.1 Ambiente de Desenvolvimento

O ambiente de desenvolvimento está sendo estruturado na engine Godot 4.7 [2], escolhida por seu suporte nativo a física 2D determinística, seu sistema de navegação (`NavigationServer2D`) e a facilidade de introspecção de dados de cena via GDScript, o que permite extrair a representação estruturada do nível sem a necessidade de processamento de imagem.

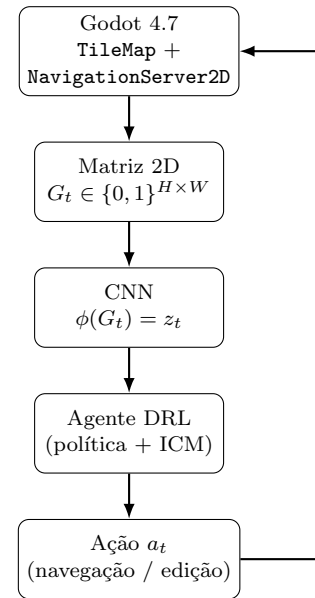


Figure 2: Pipeline proposto: extração da matriz do nível, processamento espacial via CNN, decisão do agente de DRL e retroalimentação da ação para o ambiente Godot.

4.2 Representação do Ambiente

Após avaliação da melhor estratégia de *input* para a IA, descartou-se o uso de imagens diretas da câmera (renderização de pixels): o processamento de texturas e iluminação adicionaria um custo computacional desnecessário à rede neural, sem agregar informação relevante para a tarefa de identificação de gargalos, que é fundamentalmente estrutural, e não visual.

A solução estabelecida é a extração estruturada do mapa na forma de uma matriz 2D (*grid*). Utilizando os recursos do Godot 4.7, mapeia-se o `TileMap` e as malhas do `NavigationServer2D` para gerar um *array* bidimensional cujos valores representam os estados do terreno:

- **0:** caminho livre e navegável;
- **1:** parede ou obstáculo rígido.

Nesta etapa do projeto arquitetural, a geração dessa matriz limpa pelo Godot foi isolada como uma “caixa preta” já funcional, permitindo dedicação integral à modelagem da IA. A codificação binária inicial ($\{0, 1\}$) foi escolhida por sua simplicidade; canais adicionais (posição do agente, distância à saída, densidade histórica de tráfego) poderão ser concatenados como planos extras da mesma matriz sem alterar a arquitetura do extrator convolucional, apenas o número de canais de entrada.

Table 2: Componentes do sistema de Aprendizado por Reforço Profundo

Componente	Estratégia no projeto
Observação (<i>State</i>)	A matriz 2D processada pela CNN, representando a topologia instantânea da fase ($z_t = \phi(G_t)$).
Ação (<i>Action</i>)	Navegação vetorial discreta e, posteriormente, a capacidade de sinalizar ou modificar tiles (adicionar/remover bloqueios).
Recompensa (<i>Reward</i>)	Combinação extrínseca/intrínseca: penaliza fortemente áreas de baixo fluxo de travessia (gargalos) e recompensa a exploração de regiões mal modeladas pelo agente.

4.3 Processamento Espacial com Redes Convolucionais

A matriz 2D gerada atua como a observação (*state*) para a rede neural. Para extrair os padrões topológicos desse ambiente espacial — como corredores estreitos, rotas críticas e densidade de obstáculos — os dados passam por uma *Deep Convolutional Network* [4] (CNN). A natureza das CNNs permite que o modelo compreenda o nível de forma geométrica, em vez de apenas ler números isolados, o que é essencial para reconhecer padrões espaciais que caracterizam um gargalo.

O projeto atual do extrator segue o padrão consolidado por trabalhos de RL sobre grades (*grid-world*), com camadas convolucionais de *stride* 1 e *kernels* pequenos (3×3), preservando a granularidade de *tile* necessária para localizar precisamente um gargalo, seguidas de uma camada totalmente conectada que produz o vetor latente $z_t \in \mathbb{R}^d$ utilizado pela política e pelo módulo de curiosidade. Optamos por não utilizar operações de *max-pooling* agressivas, que tenderiam a borrar exatamente as estruturas finas (corredores de largura igual a um *tile*) que o sistema precisa detectar.

4.4 Agente de Deep Reinforcement Learning

A tomada de decisão e a exploração do mapa utilizam uma arquitetura de Aprendizado por Reforço Profundo. O modelo é construído com base em conceitos avançados da literatura, unindo *Curiosity-Driven Exploration* [5] com as bases do PCGRL [3]. A Tabela 2 resume os três componentes centrais do sistema segundo a formalização apresentada na Seção 3.

Quanto ao algoritmo de otimização de política, optamos por explorar *Proximal Policy Optimization* (PPO) [6], em vez de métodos baseados em valor como *Deep Q-Networks*. Essa escolha se justifica por três razões: (i) o espaço de ações proposto na Seção 3.4 é naturalmente discreto, mas será estendido a variações contínuas de intensidade de edição em fases futuras, e o PPO acomoda ambos os casos sem mudança estrutural; (ii) o PPO é conhecido por sua estabilidade de

treinamento em cenários com recompensas esparsas e ruidosas — exatamente o cenário esperado ao combinar os termos extrínseco e intrínseco da Equação (2); e (iii) sua natureza *on-policy* simplifica a integração com o módulo de curiosidade, que depende de trajetórias recentes e consistentes com a política atual para que o erro de predição do modelo dinâmico seja informativo.

4.5 Integração de Dados e Pipeline de Treinamento

A integração entre o ambiente Godot e o ambiente de treinamento (Python) está planejada por meio de um protocolo de comunicação por sockets locais, no qual o Godot exporta, a cada *step* de simulação, a matriz G_t serializada e recebe de volta a ação a_t escolhida pela política. Essa separação de processos permite que o treinamento seja acelerado por hardware dedicado (GPU) sem sobrecarregar o laço de renderização e física do motor de jogo, e permite, no futuro, paralelizar múltiplas instâncias do ambiente para acelerar a coleta de trajetórias, uma prática comum em *pipelines* de RL profundo.

5 Estado Atual da Implementação

Com a comunicação base e o escopo da IA (matriz 2D $\rightarrow$ CNN $\rightarrow$ DRL) validados e definidos, o projeto avança para a estruturação formal do aprendizado de máquina. Até o momento:

- a extração da matriz 2D a partir do *TileMap* e do *NavigationServer2D* foi implementada e isolada como módulo independente, funcionando de forma estável para os níveis de teste utilizados durante o desenvolvimento;
- o desenho geral do pipeline *CNN + DRL* (Seção 4) foi definido e revisado frente à literatura de PCGRL e de exploração por curiosidade;
- a formalização do problema como MDP (Seção 3), incluindo a proposta inicial da função de recompensa híbrida, foi concluída nesta etapa do relatório; e
- o treinamento do agente ainda não foi iniciado — os itens acima constituem a base arquitetural sobre a qual os experimentos descritos na Seção 6 serão conduzidos.

6 Metodologia de Avaliação Proposta

Como o treinamento do agente ainda está em fase de estruturação, esta seção descreve a metodologia de avaliação planejada para os próximos ciclos do projeto, e não resultados experimentais já obtidos.

6.1 Métricas

Propomos as seguintes métricas para avaliar a capacidade do agente de identificar gargalos:

- **Precisão de detecção:** fração das células sinalizadas pelo agente como gargalo que coincidem com gargalos identificados por especialistas humanos em um conjunto de níveis de referência anotados manualmente;

- **Cobertura (*recall*)**: fração dos gargalos anotados manualmente que são efetivamente sinalizados pelo agente;
- **Entropia do mapa de visitação**: dispersão espacial das trajetórias do agente ao longo de um episódio, utilizada como *proxy* para verificar se a exploração guiada por curiosidade está de fato cobrindo regiões de baixo fluxo, e não apenas reforçando rotas já bem conhecidas; e
- **Redução de gargalos pós-edição**: para a fase futura de ações de edição (Seção 3.4), a variação do fluxo de travessia $\tau(v)$ antes e depois das intervenções sugeridas pelo agente.

6.2 Baselines

Como referência de comparação, planejamos incluir:

- um agente aleatório, que executa ações de navegação uniformemente ao acaso, para calibrar o piso de desempenho das métricas acima;
- um agente de *pathfinding* clássico (busca A* sobre o grafo de navegação), que representa a linha de base “sem aprendizado” e permite isolar o ganho específico atribuível ao componente de DRL; e
- uma variante do próprio agente proposto sem o termo intrínseco de curiosidade ($\beta = 0$ na Equação (2)), para medir a contribuição isolada da exploração guiada por curiosidade em relação à recompensa puramente extrínseca.

6.3 Protocolo Experimental

O protocolo planejado consiste em treinar cada configuração do agente em um conjunto fixo de níveis de treinamento, avaliando periodicamente em um conjunto de validação de níveis não vistos durante o treinamento, de forma a verificar a generalização da política para topologias distintas das utilizadas no treinamento – um requisito essencial para que a ferramenta seja útil em cenários reais de *level design*, nos quais o agente deve avaliar níveis inéditos, e não apenas os níveis sobre os quais foi treinado. Cada configuração será executada com múltiplas sementes aleatórias para permitir a análise de variância entre execuções, prática recomendada na literatura de aprendizado por reforço [9] dada a sensibilidade conhecida desses métodos à inicialização e à ordem de amostragem das trajetórias.

7 Discussão: Limitações e Riscos

Ainda que a arquitetura proposta seja promissora, alguns riscos metodológicos merecem discussão explícita nesta etapa do projeto.

Risco de *reward hacking*. A combinação de recompensa extrínseca e intrínseca (Equação (2)) introduz o risco clássico de que o agente explore artefatos da própria função de recompensa em vez de resolver o problema pretendido – por exemplo, permanecer em regiões cujo modelo dinâmico é intencionalmente difícil de prever (como áreas com física

caótica), gerando recompensa intrínseca alta sem corresponder a um gargalo real de *design*. Mitigar esse risco exigirá normalização cuidadosa do termo r_t^{int} e, possivelmente, seu decaimento ao longo do treinamento, à medida que o modelo dinâmico se torna mais preciso.

Generalização entre temas visuais e topológicos.

Como a representação de estado é puramente estrutural (matriz binária), espera-se melhor generalização entre temas visuais distintos do que abordagens baseadas em pixels; entretanto, a generalização entre topologias estruturalmente muito diferentes das vistas em treinamento (por exemplo, níveis com geometria não ortogonal) ainda não foi avaliada e é um risco em aberto.

Custo computacional do treinamento distribuído.

A separação entre o processo do Godot e o processo de treinamento (Seção 4) introduz sobrecarga de comunicação que pode se tornar um gargalo de desempenho – de forma irônica, o próprio pipeline de treinamento pode sofrer do tipo de problema que o agente é treinado para detectar no nível do jogo. Essa sobrecarga precisará ser medida empiricamente antes de escalar o número de ambientes paralelos.

Validação com especialistas humanos. As métricas de precisão e cobertura propostas na Seção 6 dependem da anotação manual de gargalos por especialistas, um processo que introduz subjetividade e que ainda precisa ser operacionalizado – por exemplo, definindo um protocolo de anotação com múltiplos anotadores e medidas de concordância entre eles.

8 Considerações Éticas e de Uso Responsável

Ferramentas de diagnóstico automatizado de *level design* levantam questões de uso responsável que merecem consideração explícita, mesmo em uma etapa inicial do projeto como a aqui relatada.

Complementaridade, não substituição, do julgamento humano. O sistema proposto tem como objetivo fornecer um sinal quantitativo adicional ao processo de *design*, e não substituir o julgamento de designers humanos sobre o que torna um nível interessante. Um gargalo identificado pelo agente pode ser, em certos contextos de *design* intencional (por exemplo, um estrangulamento tático em um mapa competitivo), uma característica desejada, e não um defeito a ser corrigido. A ferramenta deve, portanto, ser utilizada como apoio à decisão, com a palavra final permanecendo com a equipe de *design*.

Viés na definição de fluidez. A métrica de fluxo de travessia $\tau(v)$ e os pesos $w(v)$ de centralidade (Seção 3.5) embutem uma definição específica – e necessariamente parcial – do que constitui um nível “fluido”. Essa definição pode não ser representativa de todos os estilos de jogo ou de todos os perfis de jogadores (por exemplo, jogadores que preferem exploração lenta a progressão rápida), e deve ser comunicada de forma transparente a quem utilizar a ferramenta, para evitar que recomendações automatizadas sejam interpretadas como verdades objetivas sobre qualidade de *design*.

Dados de jogadores reais. Na fase atual do projeto, todo o treinamento ocorre em simulação, sem coleta de dados de jogadores humanos. Caso etapas futuras incorporem dados de *playtesting* real – por exemplo, para calibrar $\tau(v)$ com telemetria de partidas humanas – será necessário um protocolo explícito de consentimento e anonimização, seguindo as práticas usuais de pesquisa com participantes humanos.

9 Trabalhos Futuros

Com a arquitetura e a formalização do problema definidas, os próximos passos do projeto incluem:

- (1) **Implementação da função de recompensa:** implementar e calibrar experimentalmente os pesos β , η e θ_τ das Equações (2)–(4), hoje definidos apenas em nível conceitual;
- (2) **Implementação do espaço de ações de edição:** estender o espaço de ações de navegação para incluir as ações de sinalização e edição de tiles descritas na Seção 3.4;
- (3) **Integração de dados:** implementar o protocolo de comunicação por sockets entre o Godot e o ambiente de treinamento descrito na Seção 4;
- (4) **Construção do conjunto de níveis de referência:** preparar o conjunto de níveis anotados manualmente necessário para as métricas de precisão e cobertura da Seção 6;
- (5) **Execução dos experimentos-piloto:** treinar as primeiras versões do agente e dos *baselines* propostos, permitindo uma primeira validação empírica – ainda que preliminar – da arquitetura descrita neste artigo; e
- (6) **Extensão para mecânicas multiplayer:** avaliar como o fluxo de travessia $\tau(v)$ deve ser redefinido quando múltiplos agentes ocupam o mesmo nível simultaneamente, disputando o mesmo espaço físico.

10 Conclusão

Este artigo relatou o andamento e a arquitetura em desenvolvimento de um agente de Aprendizado por Reforço Profundo para identificação e correção de gargalos de *level design* em jogos 2D construídos na engine Godot. A representação do ambiente como matriz 2D, processada por uma CNN e consumida por um agente de DRL inspirado em PCGRL e em exploração guiada por curiosidade, oferece um caminho concreto para ir além do *pathfinding* tradicional. Formalizamos o problema como um Processo de Decisão de Markov, propusemos uma função de recompensa híbrida extrínseca/intrínseca e delineamos uma metodologia de avaliação – com métricas, *baselines* e protocolo experimental – para os próximos ciclos do projeto. Discutimos também os riscos metodológicos mais relevantes, em particular o risco de *reward hacking* inerente ao uso de recompensas intrínsecas. Os trabalhos futuros mais imediatos – implementação da função de recompensa, do espaço de ações de edição e da integração de dados – consolidarão a base necessária para os primeiros experimentos-piloto de treinamento do agente.

Acknowledgments

Agradecemos ao Prof. Augusto Baffa pela orientação ao longo do desenvolvimento deste projeto, no âmbito da disciplina Projeto Orientado IV (INF1824) da PUC-Rio.

References

- [1] Pierre Le Pelletier De Woillemont, Rémi Labory, and Vincent Corruble. 2022. Automated Play-Testing Through RL Based Human-Like Play-Styles Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 18.
- [2] Godot Engine Contributors. 2024. Godot Engine – Free and Open Source 2D and 3D Game Engine. <https://godotengine.org>.
- [3] Ahmed Khalifa et al. 2020. PCGRL: Procedural Content Generation via Reinforcement Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 16.
- [4] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [5] Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. 2017. Curiosity-Driven Exploration by Self-Supervised Prediction. In *Proceedings of the International Conference on Machine Learning (ICML)*.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [7] Alessandro Sestini et al. 2022. Automated Gameplay Testing and Validation with Curiosity-Conditioned Proximal Trajectories. *IEEE Transactions on Games* 16, 1 (2022), 113–126.
- [8] Noor Shaker, Julian Togelius, and Mark J. Nelson. 2016. *Procedural Content Generation in Games*. Springer.
- [9] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (second ed.). MIT Press.